

Large Language Model(LLM)

오늘날 우리는 검색 엔진에서 정보를 찾고, 스마트 스피커에게 말을 걸고, 번역 앱을 통해 언어 장벽을 넘는 일이 일상이 되었습니다. 이 모든 뒤에는 LLM(Large Language Model, 대규모 언어 모델)이라는 기술이 자리 잡고 있습니다. LLM은 방대한 양의 텍스트 데이터를 학습하여 마치 사람처럼 문장을 이해하고 생성할 수 있는 인공지능 모델입니다.

이번 장에서는 LLM을 이해하는 데 필수적인 네 가지 핵심 개념인 언어 모델(Language Model), 토큰(Token), 컨텍스트 윈도우(Context Window), 그리고 온도(Temperature)를 차례대로 살펴보고, 이러한 LLM에는 어떠한 한계점이 있는지에 대하여 전반적으로 살펴보도록 하겠습니다.

▼ 1. LLM 개념

LLM은 이름 그대로 대규모 언어 모델(Large Language Model)의 약자입니다. 대규모 언어 모델이란, 인간의 언어를 이해하고 생성하도록 학습된 인공지능 모델이라는 의미를 갖고 있습니다. 대규모 언어 모델은 인터넷 웹 상에서의 문서, 혹은 도서, 코드, 대화 데이터와 같은 데이터들을 거대한 규모의 텍스트 데이터로 하여금 학습하여 언어의 패턴과 구조를 통계적으로 학습한 모델입니다.

근래 서비스되고 있는 LLM은 트랜스포머(Transformer)라는 이름의 신경망 구조를 기반으로 하고 있습니다. 트랜스포머 신경망은 문장 안의 단어들이 서로 어떤 관계를 갖는지를 계산하는 셀프 어텐션(Self-Attention) 매커니즘을 사용합니다.

이를 이용하여 긴 문맥(Context Window)을 고려하면서 문장의 의미를 상대적으로 정확하게 처리할 수 있는 것입니다. 모델 내부에는 수십 억(億)개에서 수십 조(兆)개의 파라미터가 존재합니다. 여기서 파라미터는 모델이 학습 과정에서 얻게된 내용이 저장되는 수단 정도로 이해해볼 수 있습니다.

▼ 1-1. Language Model

LLM에는 언어 모델(Language Model)이 포함되어 있습니다. 언어 모델이 무엇인지를 이해해보기 위해서, 우리가 실생활에 사용하는 스마트폰의 자동완성 기능을 생각해보도록 하겠습니다. 우리가 핸드폰을 바꾸고 따로 설정을 하지 않으면 카카오톡이나 문자 메시지를 입력할때 현재 입력한 단어 다음으로 입력될 가능성이 가장 높은 단어의 후보들이 자동으로 제시되는 기능이 자동으로 적용됩니다. 가령, "오늘 날씨가" 까지 입력한다면 "좋다", "흐리다", "춥다", "덥다" 와 같은 단어들이 추천되는 방식입니다.

이것이 언어 모델의 가장 근본적인 동작원리입니다. 언어 모델이란, 주어진 텍스트 시퀀스(Sequence)를 바탕으로 다음에 등장할 단어 또는 토큰(Token)이 무엇인지 확률적으로 예측하도록 설계된 모델을 의미합니다. 이를 다시 살펴보자면 언어모델은 "어떤 단어 뒤에 어떤 단어가 올 가능성이 높은가"를 거대한 텍스트 데이터를 통해 학습한 확률 분포(Probability Distribution)으로 볼 수 있는 것입니다.

초기의 언어 모델들은 N-gram이라는 방식을 주로 사용하였습니다. N-gram이란 연속된 N개의 단어를 하나의 단위로 보고, 이전 N-1개의 단어를 보고 다음 단어를 예측하는 통계적 방법입니다. 예를 들어 "나는 밥을"이라는 두 단어를 보고 "먹었다"가 올 확률이 가장 높다고 예측하는 방식입니다. 이러한 통계적 접근은 구현이 단순하다는 장점이 있었으나, 훈련 데이터에 존재하지 않는 단어 조합이 등장하면 확률을 계산하지 못하는 희소성 문제와, 문장이 길어질수록 앞뒤 문맥을 충분히 반영하지 못하는 한계가 있었습니다.

이러한 한계를 극복하기 위한 연구가 이어지던 중, 2010년대 초반 딥러닝(Deep Learning) 기술이 자연어 처리(Natural Language Processing, NLP) 분야에 도입되면서 언어 모델의 패러다임이 변화하게 되었습니다. 그러던 와중 2017년에 Google에서 논문을 통해 트랜스포머 아키텍처를 발표하게 된 것입니다.

트랜스포머 아키텍처에서 이야기하는 이야기는 이렇습니다. 이전까지 주류였던 순환 신경망(Recurrent Neural Network, RNN) 계열의 모델들은 단어를 순서대로 하나씩 처리하는 구조적 한계로 인해, 문장이 길어질수록 앞부분의 정보를 점차 잊어버리는 장기 의존성 문제(long-term dependency problem)와 병렬 처리가 불가능하여 학습에 매우 긴 시간이 소요되는 문제를 안고 있었습니다.

트랜스포머 아키텍처에서는 이런 문제를 셀프 어텐션(Self-Attention)이라는 매커니즘을 통해 해결

했습니다. 셀프 어텐션이란, 문장 내의 모든 단어가 다른 모든 단어와의 관계를 동시에 계산하는 방식입니다. 예를 들어 “어떤 남자가 그의 어머니를 찾아갔는데, 그녀는 집에 없었다”라는 문장이 있다고 하면, 문장에서 “그녀”가 “어머니”를 가리킨다는 사실을 파악하기 위해서는 문장 내의 멀리 떨어진 단어들 사이의 관계를 이해해야 합니다. 셀프 어텐션은 입력 문장 전체를 한 번에 바라보면서 각 단어가 다른 단어들에 얼마나 주의를 기울여야 하는가에 대한 가중치를 계산합니다. 트랜스포머 아키텍처는 순차적 처리가 아닌 병렬 처리(parallel processing)가 가능하기 때문에, 막대한 양의 데이터를 훨씬 빠르게 학습할 수 있게 되었습니다.

트랜스포머 아키텍처의 등장 이후 해당 아키텍처를 기반으로 모델의 규모, 즉 매개변수(Parameter)의 수와 학습 데이터의 양을 대폭 늘렸을 때 성능이 비약적으로 향상된다는 내용을 알게 되었습니다. 이 원리를 바탕으로 탄생한 것이 바로 오늘날의 대규모 언어 모델(LLM)입니다. OpenAI의 GPT 시리즈, Google의 Gemini, Meta의 LLaMA, 그리고 Anthropic의 Claude 등이 모두 이 범주에 속합니다.

▼ 1-2. Token

이전 장에서 우리는 언어 모델이 다음에 올 단어를 예측하는 모델이라고 정리하였습니다. 실제로 언어 모델(LLM)은 이 단어를 예측하기 위해서 단어보다 조금 더 작은 단위를 사용합니다. 이 단위를 우리는 토큰(Token)이라고 일컫습니다. 컴퓨터가 언어를 처리하려면 텍스트를 수치 데이터로 변환해야 합니다. 언어를 수치로 표현하기 위한 가장 첫 번째 단계가 바로 텍스트를 일정한 단위로 분할하는 토큰화(Tokenization)이며, 이 과정에서 만들어지는 각 조각이 토큰인 것입니다.

즉, 토큰이란 자연어 처리(NLP) 분야에서 텍스트를 구성하는 가장 작은 처리 단위로, 모델이 학습하고 추론하는 데 있어 언어의 기본 원자(Atom)와 같은 역할을 합니다. 토큰화와 토큰을 이해하기 위하여 우리가 책 한 권을 분석하는 과정을 생각해보도록 하겠습니다. 책 전체를 동시에 한꺼번에 분석하기는 어렵기 때문에, 자연스럽게 문단으로 나누고, 문단을 문장으로, 문장을 단어로, 단어를 음절로 나누어 가며 분석하는 방식을 사용합니다. 토큰화도 이와 유사한 방식으로 텍스트를 작은 단위로 분할하는 과정입니다. 언어 모델에서의 토큰은 우리가 일상적으로 생각하는 단어와 반드시 일치하지는 않습니다. 토큰은 경우에 따라 단어 전체가 되기도 하고, 단어의 일부(접두사, 어간, 접미사)가 되기도 하며, 때로는 하나의 문자나 구두점이 하나의 토큰이 되기도 합니다.

토큰화 방식은 크게 3가지 방식으로 분류해 볼 수 있습니다.

▼ 1-2-1. 단어 기반 토큰화(Word-level Tokenization)

단어 기반 토큰화(Word-level Tokenization)는 텍스트를 단어 단위로 분리하여 토큰(Token)을 만드는 방법을 의미합니다. 토큰(Token)은 모델이 처리하는 최소 단위의 텍스트 조각을 의미하며, 단어 기반 토큰화에서는 일반적으로 공백(space)이나 문장부호를 기준으로 문장을 분리합니다. 가령 "나는 운동을 좋아한다" 이라는 문장이 있다면 단어 기반 토큰화 결과는 ["나는", "운동을", "좋아한다"] 와 같이 각 단어가 하나의 토큰이 됩니다.

참고로 한국어와 같은 경우는 어근에 조사나 어미가 붙는 교착어에 해당하기 때문에 단순히 띄어쓰기로만 나누면 AI가 언어를 학습하는 것에 효율이 크게 떨어집니다. 따라서 일반적으로는 단어 기반 토큰화가 아니라 형태소(形態素)기반 토큰화(KoNLPy, MeCab 등)를 주로 사용합니다.

단어 기반 토큰화 방식의 장점은 구조가 매우 단순하고 사람이 이해하기 쉽다는 것입니다. 텍스트를 그대로 단어 단위로 나누기 때문에 토큰의 의미가 직관적으로 유지됩니다. 또한 토큰이 단어 자체이기 때문에 자연어 처리 초기 단계의 모델에서는 구현과 해석이 비교적 용이합니다.

그러나 단어 기반 토큰화는 몇 가지 중요한 한계를 가진다. 가장 큰 문제는 어휘 집합(Vocabulary) 크기입니다. 언어에는 매우 많은 단어가 존재하고, 새로운 단어도 계속 등장합니다. 이 때문에 모든 단어를 사전에 등록하려면 어휘 크기가 비대해집니다. 또한 학습 데이터에 없던 단어가 등장하면 이를 처리하지 못하는 OOV(Out Of Vocabulary) 문제가 발생할 수 있습니다. 가령 영어를 기준으로 모델이 "play"라는 단어는 알고 있지만 "playing", "played", "player" 같은 변형 단어를 모르면 새로운 단어로 인식하게 됩니다.

▼ 1-2-2. 문자 기반 토큰화(Character-level Tokenization)

문자 기반 토큰화(Character-level Tokenization)는 텍스트를 문자 단위로 분리하여 토큰(Token)을 생성하는 방법입니다. 문자 기반 토큰화 방식에서는 단어가 아니라 각각의 문자(Character)가 하나의 토큰이 됩니다. 예를 들어 "안녕하세요"라는 문장이 있다면 ["안", "녕", "하", "세", "요"] 처럼 각각의 글자가 하나의 토큰이 되는 방식입니다.

문자 기반 토큰화의 가장 큰 특징은 어휘 집합(Vocabulary)이 매우 작은 것입니다. 단어 기반 토큰화는 수만 개에서 수십만 개의 단어가 필요하지만, 문자 기반 토큰화는 알파벳, 숫자, 특수문

자 정도만 포함하면 되므로 수십 개에서 수백 개 정도의 토큰만으로도 모든 문장을 표현할 수 있습니다. 또한 어떤 새로운 단어가 등장하더라도 문자 단위로 분해할 수 있기 때문에 OOV(Out Of Vocabulary) 문제가 상대적으로 거의 발생하지 않습니다.

그러나, 단어 하나를 표현하기 위해 여러 개의 문자를 사용해야 하므로 문장의 길이가 매우 길어지는 문제가 발생합니다. 예를 들어 internationalization 같은 단어는 단어 토큰 하나로 표현될 수 있지만 문자 기반 토큰화에서는 20개 이상의 토큰이 필요합니다. 이렇게 토큰 수가 증가하면 모델이 처리해야 할 계산량이 커지고 문맥(Context)을 이해하기도 어려워지는 한계점이 있습니다.

▼ 1-2-3. 부분어 기반 토큰화(Subword Tokenization)

부분어 기반 토큰화(Subword Tokenization)방식은 텍스트를 단어보다 작은 단위인 부분어(Subword) 단위로 분리하여 토큰(Token)을 만드는 방법입니다. 부분어 기반 토큰화 방식은 단어 기반 토큰화와 문자 기반 토큰화의 장점을 결합하기 위해 출시됐습니다.

단어 기반 방식은 의미 단위가 명확하지만 어휘 집합(Vocabulary)이 매우 커지고 OOV(Out Of Vocabulary) 문제가 발생할 수 있었습니다. 반면 문자 기반 방식은 어휘 크기는 작지만 토큰 수가 지나치게 많아져 모델의 계산량이 증가할 수 있었습니다. 부분어 토큰화방식은 이러한 두 문제 사이의 균형을 맞추는 방식입니다.

부분어 토큰화에서는 자주 등장하는 단어는 그대로 하나의 토큰으로 유지하고, 드물게 등장하는 단어는 더 작은 조각으로 분해합니다. 예를 들어 "unhappiness"라는 단어는 ["un", "happi", "ness"]와 같이 여러 개의 부분어로 나눌 수 있습니다. 한국어를 기준으로 보면 데이터과학자라는 단어는 ["데이터", "과학", "자"]처럼 분리될 수 있는 것입니다. 이렇게 하면 모델은 새로운 단어가 등장하더라도 이미 학습한 부분어 조합으로 단어를 이해할 수 있습니다.

대표적인 부분어 토큰화 알고리즘으로는 Byte Pair Encoding(BPE), WordPiece, SentencePiece 등이 있습니다. BPE는 가장 자주 등장하는 문자 쌍을 반복적으로 결합하여 토큰을 생성하는 방식이고, WordPiece는 확률 기반으로 가장 효율적인 부분어 단위를 선택합니다. SentencePiece는 공백까지 포함한 문자 단위에서 시작해 부분어를 학습하는 방식입니다.

근래 사용되는 대부분의 LLM에서는 부분어 기반 토큰화를 사용합니다. 실제로 GPT 계열 모델은 BPE 계열 토큰화를 사용하고 있습니다. 이러한 방식 덕분에 모델은 어휘 크기를 수만 개 수준으로 유지하면서도 새로운 단어를 안정적으로 처리할 수 있습니다.

▼ 1-2-4. API에서의 토큰

LLM을 API로 사용할 때의 비용은 일반적으로 처리된 토큰의 수를 기준으로 산정됩니다. 즉, 입력 프롬프트와 모델의 응답에 포함된 총 토큰 수가 곧 비용과 직결됩니다. 또한 토큰의 수는 모델이 한 번에 처리할 수 있는 텍스트의 양, 즉 다음 절에서 살펴볼 컨텍스트 윈도우의 크기와의 직접적으로 연결됩니다. 영어의 경우 평균적으로 약 1개의 토큰이 0.75개의 단어에 해당하며, 한국어의 경우 형태소 단위로 분할되는 특성상 영어보다 더 많은 토큰이 필요한 경우가 많습니다.

또한 토큰화 방식 자체가 LLM의 성능에도 영향을 미칩니다. 예를 들어 LLM이 수학 연산이나 철자를 세는 작업에서 어려움을 겪는 경우가 종종 있는데, 이는 숫자나 단어가 우리가 생각하는 방식이 아닌 불연속적인 토큰의 조합으로 표현되기 때문입니다.

▼ 1-3. Context Window

언어 모델이 어떻게 텍스트를 처리하는지 이해했다면, 이제는 모델이 한 번에 얼마나 많은 텍스트를 처리할 수 있는지를 결정하는 컨텍스트 윈도우(Context Window)의 개념을 살펴보아야 합니다. 컨텍스트 윈도우를 이해하기 위한 가장 좋은 비유는 단기 기억력(short-term memory)입니다. 우리가 대화를 할 때, 방금 전에 나눈 이야기는 생생하게 기억하지만, 한 시간 전에 나눈 이야기는 세부 내용이 흐릿해집니다. LLM도 이와 유사하게 동작합니다. 모델은 주어진 입력 전체를 동시에 참고하여 출력을 생성하는데, 이때 '한 번에 참고할 수 있는 텍스트의 최대 분량'이 바로 컨텍스트 윈도우입니다.

이를 다시 정리해보자면, 컨텍스트 윈도우란 LLM이 텍스트를 이해하거나 생성할 때 입력으로 받아들여 동시에 처리할 수 있는 최대 토큰의 수를 의미합니다. 컨텍스트 윈도우는 앞서 정리한 토큰 단위로 측정됩니다. 예를 들어 어떤 모델의 컨텍스트 윈도우가 4,096 토큰이라고 한다면, 해당 모델은 한 번의 처리 과정에서 최대 4,096개의 토큰으로 이루어진 텍스트만을 입력으로 받을 수 있는 것입니다. 이 한도를 초과하는 텍스트가 입력될 경우, 모델은 윈도우 범위를 벗어난 이전 내용을 더 이상 기억하지 못하며, 이는 마치 대화의 앞부분을 망각한 것과 같은 상태가 되는 것입니다.

컨텍스트 윈도우가 기술적으로 어떻게 구현되는지를 이해하려면 트랜스포머의 셀프 어텐션 메커니즘을 다시 상기할 필요가 있습니다. 셀프 어텐션은 입력으로 들어온 모든 토큰 사이의 관계를 계산합니다. 구체적으로는 각 토큰에 대해 쿼리(Query), 키(Key), 값(Value)라는 세 가지 벡터를 생성하고, 현재 토큰의 쿼리와 모든 다른 토큰의 키 사이의 유사도를 계산하여 어텐션 가중치(attention weight)를 구합니다. 이 가중치를 각 토큰의 값에 곱하여 합산함으로써, 현재 토큰의 문맥을 반영한 새로운 표현을 생성합니다. 이 연산의 계산량은 입력 토큰 수의 제곱에 비례하여 증가합니다. 즉, 입력 토큰이 두 배로 늘어나면 계산량은 네 배가 됩니다. 이것이 컨텍스트 윈도우의 크기를 무한정 늘리지 못하는 근본적인 이유입니다.

컨텍스트 윈도우의 크기는 LLM 응용 프로그램의 설계에 직접적인 영향을 미칩니다. 컨텍스트 윈도우가 크면 클수록 더 긴 문서를 분석하거나, 더 오래된 대화 내용을 참조하거나, 더 많은 예시를 한꺼번에 제공하는 인컨텍스트 학습(In-Context Learning)이 가능해집니다.

초기 GPT-3 기반의 ChatGPT가 처음 등장했을 때 컨텍스트 윈도우는 약 2,048 토큰(약 1,500 단어)정도였습니다. 이후 기술의 발전과 함께 컨텍스트 윈도우의 크기는 크게 성장하였고, 2025년 기준 Google의 Gemini 모델은 최대 200만 토큰, 즉 수천 페이지 분량의 문서를 한 번에 처리할 수 있는 수준에 도달하였습니다.

그러나 컨텍스트 윈도우가 크다고 무조건 좋은 것은 아닙니다. 연구에 따르면 입력된 정보의 양이 지나치게 많아지면 모델이 중간에 위치한 정보를 효과적으로 활용하지 못하는 경향이 있으며, 처리에 소요되는 계산 비용과 시간도 크게 증가합니다. 이러한 한계를 보완하기 위한 기술로 LangChain에서도 활발히 다루는 RAG(Retrieval-Augmented Generation, 검색 증강 생성)가 있으며, 이는 이후에 다시 살펴보도록 하겠습니다.

▼ 1-4. Temperature

온도(Temperature)는 LLM의 출력 결과를 제어하는 역할을 수행하는 매개변수입니다. 소설을 쓸 때는 같은 상황을 수십 가지의 다른 방식으로 묘사할 수 있으며, 오히려 예측할 수 없는 표현이 글을 더욱 풍성하게 만들기도 합니다. LLM의 온도는 얼마나 다양하고 창의적인 출력을 생성할 것인가를 조절하는 매개변수입니다.

온도의 작동 방식을 이해하기 위해서는 LLM이 어떻게 다음 토큰을 선택하는지를 먼저 살펴보아야 합니다. LLM은 입력 텍스트를 처리한 후, 어휘 사전에 존재하는 모든 토큰 각각에 대해 다음 토큰으로 얼마나 적합한가를 나타내는 원시 점수를 계산합니다. 이 원시 점수를 로짓(logit)이라고 일컫습니다. 로짓은 정규화되지 않은 실수 값이므로 확률처럼 직접 사용하기 어렵습니다. 따라서 이 로짓 값들을 소프트맥스(Softmax) 함수에 통과시켜 총합이 1이 되는 확률 분포로 변환합니다. 따라서 이 로짓 값들을 소프트맥스 함수에 통과시켜 총합이 1이 되는 확률 분포로 변환합니다.

소프트맥스 함수는 각 로짓 값에 지수 함수를 적용한 뒤 전체 합으로 나누는 방식으로, 가장 높은 로짓 값을 갖는 토큰이 가장 높은 확률을 부여받으며, 낮은 로짓 값을 갖는 토큰도 0이 아닌 작은 확률을 부여받게 됩니다. 모델은 이렇게 만들어진 확률 분포를 바탕으로 다음 토큰을 선택하는 방식입니다.

식당에서 메뉴를 고르는 상황을 상정해보겠습니다. 메뉴를 선택할 때 온도값을 적용할 수 있다고 한다면, 온도가 매우 낮을 때(0에 가까울 때)는 마치 '항상 가장 좋아하는 음식만 주문하는 사람'과 같이 주문합니다. 매번 방문할 때마다 동일한 메뉴를 선택하므로, 예측 가능하고 실수가 없습니다. 반면에 온도가 매우 높은 경우에는 '오늘은 완전히 낯선 음식을 먹어보겠다'고 결심한 사람과 같습니다. 평소에는 잘 시키지 않던 메뉴를 선택하게 되어 다양하고 창의적인 경험을 할 수 있지만, 간혹 입맛에 맞지 않는 음식이 나올 수도 있습니다.

이런 온도값은 프로젝트별로 설정하는 방식이 전부 달라집니다. 즉, 규격화되어있는 방법이 없다는 것입니다. 온도의 적절한 값은 수행하려는 작업의 성격에 따라 달라집니다. 정확한 사실 정보를 답변해야 하는 질의응답 시스템, 코드 생성, 데이터 추출, 수학 문제 풀이 등 정답이 명확하게 존재하는 작업에는 낮은 온도(0.0 ~ 0.3)를 사용하여 모델이 학습을 통해 가장 확실하게 알고 있는 정보를 출력하도록 유도하는 것이 바람직하고, 이메일 초안 작성, 광고 문구 생성, 아이디어 브레인스토밍, 이야기 창작 등 다양하고 창의적인 출력이 요구되는 작업에는 높은 온도(0.7 ~ 1.2 혹은 그 이상)를 사용하여 예상치 못한 표현과 다채로운 결과를 유도할 수 있습니다.

▼ 2. LLM의 한계

앞선 절에서 우리는 LLM이 어떻게 동작하는지, 그리고 그 핵심적인 구성 요소들이 무엇인지를 살펴보았습니다. 언어 모델이 텍스트를 토큰 단위로 처리하고, 컨텍스트 윈도우라는 범위 안에서 문맥을 파악하며, 온도라는 매개변수로 출력의 다양성을 조절한다는 사실을 정리했습니다.

LLM도 특정 유형의 작업이나 상황에서는 신뢰하기 어려운 결과를 내놓거나 아예 답변 자체가 불가능한 경우가 있습니다. 이번 절에서는 LLM이 가진 대표적인 세 가지 한계인 환각(Hallucination), 지식 마감(Knowledge Cutoff), 그리고 외부 시스템 접근 불가에 대해 차례대로 살펴보도록 하겠습니다.

▼ 2-1. Hallucination

LLM을 처음 사용해본 사람이라면 한 번쯤은 이런 경험을 해보셨을 것입니다. 분명히 자신 있어 보이는 어조로 답변을 내놓았는데, 나중에 알고 보니 그 내용이 사실무근인 내용이었던 것입니다. 언론에서도 다루었던 ChatGPT에게 '세종대왕의 맥북프로 던짐 사건'을 물어보았을 때 모델이 마치 실제 있었던 역사적 사건인 것처럼 상세하게 설명을 장황히 늘어놓았던 사례도 있습니다. 이처럼 LLM이 실제로 존재하지 않는 정보나 사실과 다른 내용을 마치 진실인 것처럼 생성하는 현상을 환각(Hallucination)이라고 합니다.

환각이라는 용어는 의학적 개념에서 빌려온 표현으로, 원래는 '외부의 자극이 없는데도 불구하고 마치 외부에서 자극이 들어온 것처럼 느끼는 현상'을 뜻합니다. LLM의 환각도 이와 유사합니다. 실제로 그러한 사실이 존재하지 않음에도 불구하고, 모델은 그것이 존재하는 것처럼 텍스트를 생성합니다.

LLM의 환각은 크게 두 가지 유형으로 분류됩니다. 첫 번째는 내재적(Intrinsic) 환각으로, 입력으로 제공된 텍스트의 내용과 모순되는 출력을 생성하는 경우입니다. 예를 들어 '삼성전자가 2024년 1월 17일에 언팩 행사를 개최했다'는 내용을 제시하고 날짜를 질문했을 때 '2024년 7월 11일'이라고 답하는 경우가 이에 해당합니다. 두 번째는 외재적(Extrinsic) 환각으로, 입력 텍스트와는 관련 없이 외부의 사실과 다른 내용을 생성하는 경우입니다. 존재하지 않는 인물의 경력, 출판되지 않은 책의 제목, 가상의 학술 논문 등을 실제처럼 제시하는 것이 이에 해당합니다.

환각이 발생하는 근본적인 원인은 LLM의 작동 방식 자체에 있습니다. 이전 장에서 정리했던 것처럼, LLM은 본질적으로 다음에 올 토큰을 확률적으로 예측하는 기계입니다. 언어 모델은 훈련 과정에서 수십억 개의 문장 패턴을 학습하였고, 이를 바탕으로 주어진 문맥에서 가장 그럴듯한 텍스트를 생성하는 것입니다.

여기서 핵심은 모델이 생성하는 텍스트의 언어적 유창성과 그 내용의 사실 여부는 전혀 별개라는 부분입니다. 모델은 자신이 생성하는 내용의 진위를 검증하는 내부 메커니즘을 갖고 있지 않습니다. 즉, 스스로 생성한 내용이 사실인지 거짓인지를 구별하지 못한 채 오직 언어적 패턴의 자연스러움만을 기준

으로 텍스트를 출력하는 것입니다.

▼ 2-1-1. 환각의 원인

LLM이 환각을 일으키는 원인은 다음과 같이 정리해볼 수 있습니다.

첫 번째 원인은 LLM이 확률 기반 예측 구조로 구성되어있기 때문입니다. LLM은 질문의 정답을 데이터베이스에서 검색하여 가져오는 방식이 아니라, 이전 문맥을 기반으로 다음에 등장할 가능성이 가장 높은 토큰(Token)을 예측하는 방식으로 문장을 생성합니다.

이 과정에서 모델은 실제 사실 여부보다는 문장이 자연스럽게 이어질 확률을 기준으로 선택을 합니다. 예를 들어 어떤 인물의 논문 제목이나 연도를 질문받았을 때 정확한 정보를 기억하지 못하더라도, 학습된 언어 패턴을 기반으로 그럴듯한 답을 만들어낼 수 있습니다. 이러한 특성 때문에 모델은 틀린 정보를 매우 자연스럽게 생성할 수 있는 것입니다.

두 번째 원인은 학습 데이터의 불완전성입니다. LLM은 인터넷 문서, 책, 코드, 기사 등 다양한 텍스트 데이터를 학습하지만 모든 정보를 완벽하게 포함할 수는 없습니다. 데이터에는 오류가 포함되어 있을 수도 있고, 특정 주제에 대한 정보가 부족할 수도 있습니다. 또한 학습 데이터는 특정 시점까지의 정보만 포함하기 때문에 최신 정보는 반영되지 않습니다. 이런 상황에서 모델이 답변을 생성해야 할 경우, 부족한 정보를 채우기 위해 추론처럼 보이는 문장 생성을 하게 되며 이것이 환각으로 나타날 수 있는 것입니다.

세 번째 원인은 지식 표현 방식의 한계입니다. LLM 내부의 지식은 데이터베이스처럼 명확한 구조로 저장되어 있는 것이 아니라 수십억 개 이상의 파라미터안에 분산 표현형태로 저장됩니다. 즉, 특정 사실이 하나의 위치에 명확하게 기록되어 있는 것이 아니라 여러 가중치(Weight)와 연결 관계 속에 퍼져 있는 형태인 것입니다. 이 때문에 모델은 어떤 정보를 정확하게 조회하기보다는 패턴을 재구성하여 문장을 생성합니다. 이러한 구조는 언어 생성에는 강점을 가지지만 사실 정확성에는 한계를 갖게됩니다.

네 번째 원인은 프롬프트와 문맥(Context)의 영향입니다. LLM은 입력된 프롬프트와 이전 대화 문맥에 강하게 영향을 받습니다. 만약 질문 자체가 잘못된 가정을 포함하고 있거나 모호한 표현을 포함하고 있다면 모델은 그 전제를 그대로 받아들여 답변을 생성할 수 있습니다. 예를 들어

존재하지 않는 연구나 사건을 질문하면 모델은 이를 검증하지 않고 설명을 만들어낼 수 있습니다. 이러한 상황에서는 질문 자체가 환각을 유도하는 구조가 됩니다.

▼ 2-2. Knowledge-cutoff

LLM의 두 번째 단계는 지식 마감일(Knowledge Cutoff)에 관한 것입니다. LLM은 인터넷의 방대한 텍스트 데이터를 바탕으로 사전 학습(Pre-training)을 수행합니다. 이 과정은 엄청난 양의 컴퓨팅 자원과 시간을 소모하기 때문에, 모델 개발사들은 특정 시점을 기준으로 데이터 수집을 마감하고 그 시점까지의 데이터만으로 모델을 학습시킵니다. 이처럼 학습 데이터가 수집된 마지막 시점을 지식 마감일(Knowledge Cutoff) 또는 훈련 데이터 종료 시점(Training Data Cutoff)이라고 합니다.

모델은 학습 데이터가 종료된 시점 이후에 발생한 사건, 새로 등장한 기술, 최신 연구 결과, 개정된 법률이나 정책, 변동된 시장 상황 등에 대해 전혀 알지 못합니다. 지식 마감일 이후의 세계는 모델에게 있어 완전한 미지의 영역입니다.

가령 "사용자가 현재 우리나라 기준금리는 얼마인가요?"라고 질문하였다고 가정해보겠습니다. 기준금리는 경제 상황에 따라 수시로 변경되는 정보입니다. 만약 모델의 지식 마감일이 6개월 전이라면, 모델은 6개월 전의 기준금리를 '현재 기준금리'로 제시하게 됩니다. 더 심각한 경우, 모델은 자신의 지식이 최신 정보가 아닌 것을 인식하지 못하고 마치 현재 유효한 정보인 것처럼 확신에 찬 어조로 답변할 수 있습니다. 선거 결과, 기업의 최신 분기 실적, 특정 인물의 현재 직책, 출시된 지 얼마 되지 않은 제품이나 서비스에 대한 정보, 최근 발생한 사건과 사고 등이 모두 같은 이유로 잘못되거나 불완전한 답변이 나올 수 있는 영역입니다.

또한 지식 마감일은 단순히 '최신 정보가 없다'는 문제에 그치지 않습니다. 모델이 자신의 지식 마감일을 인식하지 못하거나 부정확하게 인식하는 경우에는 앞서 다룬 환각 현상과 결합하여 더욱 심각한 문제를 일으킬 수 있습니다. 지식 마감일 이후의 정보에 대해 질문을 받았을 때, 모델은 세 가지 중 하나로 반응합니다. 모른다고 솔직하게 인정하거나, 낡은 정보를 그대로 제공하거나, 혹은 최악의 경우 관련 없는 패턴을 조합하여 사실처럼 들리는 잘못된 정보를 생성하는 것입니다. 세 번째 반응이 바로 지식 마감일과 환각이 결합된 형태입니다. 이를 방지하기 위해 근래의 LLM들은 지식 마감일이 지난 정보에 대해서는 명시적으로 자신의 한계를 알리도록 학습은 되어있지만 완벽하게 동작하지는 않습니다.

▼ 2-3. 외부 시스템의 접근 불가

LLM은 기본적으로 텍스트를 입력받아 텍스트를 출력하는 폐쇄된 시스템입니다. 추론 시점 (Inference Time)에 모델 자체는 외부 세계와 단절되어 있으며, 오직 컨텍스트 윈도우에 담긴 텍스트만을 참고하여 응답을 생성합니다. 이 말은 곧 모델이 스스로 특정 조직의 데이터베이스를 조회하거나, 웹 페이지를 방문하거나, 파일을 읽거나, 외부 API를 호출하거나, 코드를 실행하는 등의 행위를 할 수 없거나 어렵다는 것을 의미합니다.

고객 서비스 챗봇을 LLM으로 구축하려는 상황을 가정해보겠습니다. 고객이 '제 주문 상태가 어떻게 되나요?'라고 질문했다고 하면, 이 질문에 정확하게 답하려면 주문 관리 데이터베이스에서 해당 고객의 주문 정보를 조회해야 합니다. 그러나 기본 상태의 LLM은 그 데이터베이스에 접근할 방법이 없습니다. 마찬가지로 '오늘 서울 날씨는 어때요?'라는 질문에 답하려면 실시간 기상 데이터를 제공하는 API에 접근해야 하며, '지금 어떤 회사의 주가가 얼마예요?'라는 질문에는 실시간 증권 정보 서비스에 접근해야 합니다. 이러한 실시간성이 요구되는 모든 작업이 기본 LLM으로는 불가능합니다.

이와 같은 외부 시스템 접근 불가의 문제는 앞서 다룬 지식 마감일 문제와도 밀접하게 연결되어 있습니다. 외부에 있는 최신 정보 소스에 접근할 수 없기 때문에, 훈련 데이터 수집이 마감된 이후의 정보를 획득하는 것이 원천적으로 불가능한 것입니다. 즉, 지식 마감일 문제는 외부 시스템 접근 불가라는 더 근본적인 한계로부터 비롯된다고도 볼 수 있습니다. 결국 LLM은 자신의 매개변수 속에 내재화된 지식만을 이용할 수 있는, 일종의 '폐쇄된 지식 저장소'에 가깝습니다.

지금까지 살펴본 세 가지 한계인 환각, 지식 마감일, 그리고 외부 시스템 접근 불가는 각각 독립적으로 발생하기도 하지만 서로 복잡하게 얽혀 나타나기도 합니다.